Fangorn I — The Mathematical Core (study sheet)

Condensed from WHITEPAPER_1.md. Definitions, algorithms, results; proofs as one-line ideas.

Flow: names → Merkle DAGs → map/log commitments → ordering → repos, citations, apps → registry (heads only) → global state → directories & the update log → reading slices, citations & history → lenses → indexers & search (bundles, bounded trees) → instantiation (MST, MMR) → agents, micro-commits, fraud proofs (appendices).

---

1. Primitives

Notation. λ security parameter; enc canonical injective encoding; $F(x_1, \ldots, x_n) := F(\mathsf{enc}(x_1, \ldots, x_n))$. $K \rightharpoonup V$ finite partial map. fail propagates to callers. $\mathsf{Get}(\alpha)$: fetch from any source, accept iff its name is α.

Def 1.1 (Binding name). $N : \{0,1\}^* \to \{0,1\}^{2\lambda}$. $\mathbf{Adv}^{\mathsf{bind}}_N(\mathcal{A}) = \Pr[\mathcal{A} \to x \neq x',\ N(x) = N(x')]$ is negligible. (= collision resistance.)

Def 1.2 (Node). $n = (v, C)$, payload $v \in \{0,1\}^*$, links $C = (c_1, \ldots, c_m)$ (ordered). Name $N(n) = N(v, C)$.

Def 1.3 (Merkle DAG). Finite node set $\mathcal{N}$ with (1) *closure*: every link names a node in $\mathcal{N}$; (2) *distinct names*.

- $n_\alpha < n_\beta$ (descends) if a path $n_\beta \to \cdots \to n_\alpha$ of ≥ 1 links exists.
- $\mathcal{N}{\downarrow}\alpha$: nodes reachable from n_α (itself a Merkle DAG, root α).

Lemma 1.4 (Acyclic; ROM). With q oracle queries, $\Pr[\text{cycle}] \leq q^2/2^{2\lambda}$. *Idea:* the first-queried node on a cycle contains the output of a later query. Needs unpredictability, not just binding.

Lemma 1.5 (A name identifies its sub-DAG). $\mathcal{N}{\downarrow}\alpha \neq \mathcal{N}'{\downarrow}\alpha \Rightarrow$ collision in $\mathcal{N} \cup \mathcal{N}'$. *Idea:* walk a path from α in both; the first point of divergence is two nodes with one name. ⇒ **content addressing**: fetching by name from untrusted hosts yields exactly $\mathcal{N}{\downarrow}\alpha$.

Def 1.6 (Map commitment Com). Finite map $\mapsto$ root of a deterministic Merkle DAG over its entries, with:

1. **binding:** $D \neq D' \Rightarrow \mathsf{Com}(D) \neq \mathsf{Com}(D')$ (w.h.p.);
2. **unique representation:** $\mathsf{Com}(D)$ depends only on D, not on edit history;
3. **interval openings:** keys totally ordered; $\mathsf{Open}(D, I) \to \pi$, $\mathsf{Check}(root, I, D', \pi) \in \{0,1\}$
   - correctness: $\mathsf{Check}(\mathsf{Com}(D), I, D|_I, \mathsf{Open}(D, I)) = 1$;
   - opening binding: no two accepted $D' \neq D''$ for one $(root, I)$, except $\mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}$.
   - $[k, k]$ also proves absence.

Def 1.7 (Log commitment LCom). Sequence $L = (l_1, \ldots, l_M) \mapsto$ root, with:

1. **binding** (root fixes M);
2. **positional openings** $\mathsf{Prove}(L, j)$, $\mathsf{Verify}(root, j, l, \pi)$; position binding except $\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$;
3. **incremental appends:** small state $\mathsf{st}(L)$ fixing M; $\mathsf{Root}(\mathsf{st}(L)) = \mathsf{LCom}(L)$, $\mathsf{Append}(\mathsf{st}(L), l) = \mathsf{st}(L \| l)$.

Follow an untrusted log: hold st, append received entries, compare Root to an authenticated root ⇒ all entries, in order.

Def 1.8 (NP relation). $R \subseteq \{0,1\}^* \times \{0,1\}^*$ with $|w| \le p(|x|)$ and $(x,w) \in R$ decidable in $\mathrm{poly}(|x|)$. $\mathcal{L}_R = \{x : \exists w, (x,w) \in R\}$. (Used by [WP2] for read conditions: statement = what is to be read; witness = evidence the reader may, e.g. a payment or a membership proof.)

Def 1.9 (Merkle clock). Merkle DAG of events; each new node links to all current roots. $a \prec b$ iff $n_a < n_b$. Gives Lamport happened-before; one writer ⇒ a chain.

Def 1.10 (Ordering service). Puts everyone's updates into one total order, with a logical clock. It keeps a sequence $\mathcal{O} = (o_1, o_2, \dots)$ of entries $o_i = (t_i, a_i, d_i, op_i)$:

- t_i clock value (assigned by the service), a_i author, d_i **destination** (the registry it is addressed to, §3), op_i operation (both chosen by the author).
- Interface: $\mathsf{Submit}(o)$ proposes an entry; $\mathsf{Read}(t)$ returns the prefix up to clock t; $\mathsf{Verify}(d, \mathcal{O}')$ checks that $\mathcal{O}'$ validly extends checkpoint d.
- Epoch $e(t) = \lfloor t/\eta \rfloor$ for a fixed length η.

Assumption O (Ordering).

1. **Agreement:** all honest parties see the same sequence, except for entries less than δ clock steps old (the finality delay).
2. **Authorship:** an efficient adversary outputs an accepted entry (t, a, d, op) where a never signed (d, op) only with negligible probability. *(The signature covers the destination, so an entry can't be redirected to another registry.)*
3. **Clock:** clock values never decrease along the sequence.

---

2. Data, citations, apps

- **Record** $r = (\theta, v)$: type, field map. **Version** $= N(\mathtt{rec}, r)$.
- **State** $M : K \rightharpoonup Rec$, $\quad \mathsf{root}(M) = \mathsf{Com}\big(\{k \mapsto N(\mathtt{rec}, M(k))\}\big)$.
- **Repo** $\rho = (P, s)$: one publisher's data. Only P writes it. It **belongs to no app and names no rule**.
- **Citation**: a record whose payload holds $ref = (\rho, i, h, k)$, the record at key k of ρ at position i, head h ($k = \perp$ cites the whole repo). Other fields are the citer's own: rank, tag, review, verdict. Naming h binds the content.
- **App** = a repo of citations, written by its **curator**, plus a **manifest** record (purpose, lenses served, where it is paid [WP2 §2.4]).
    - Present = cite; upgrade = cite a later position; withdraw = delete the citation. Each an ordinary update.
    - Holds no one else's data, dictates no schema, admits no one. No publisher needs an app. Apps may cite apps.
- **Validator** = a publisher whose citations carry a verdict. Evidence: the data itself, a succinct proof, or an attestation; exactly as strong as that evidence. Single-record rules can also be checked at read time by a lens.
- **Moderation by a group:** make the curator a program on the ordering (§5) that writes when its rule is met (e.g. a threshold of signatures).

- A publisher can't stop a citation (like a link), but can withhold payment from the citer [WP2 §2.4] and seal data [WP2].
- **Hosts** keep or drop what they choose (§7); drop only names no served state reaches. Unserved data ⇒ reads fail, never lie.

---

3. History and the registry

Commit $\gamma = (e, parent, root)$; **head** $= N(\gamma)$; M_h = state of h, $M_\perp = \emptyset$. A repo's commits are a one-writer Merkle clock ⇒ a chain. The clock alone can't say a head is the **latest**; the ordering does.

Parameters, fixed at deployment: identifier ι (chain id + contract address); window $W \geq 1$ epochs.

Which entries it reads. Registry ι runs over $\mathcal{O}_\iota$ (destination $d = \iota$), entries written $o = (t, a, op)$. By Authorship, nobody moves an entry between registries. Values signed apart from the ordering (micro-commits, [WP2] terms) name ι themselves.

Def 3.1 (Registry state). $R = (\mathcal{H}, \mathcal{U})$:

- $\mathcal{H}(\rho) = (pos, h)$, $(0, \perp)$ if never written;
- $\mathcal{U}$ = the **update log**, one entry per valid update.

One operation.

```
Apply(R, o)          o = (t, a, update(s, e, parent, root))
1  P ← a;  (i, h) ← H(P,s);  require parent = h ∧ e(t)−W ≤ e ≤ e(t)
2  h' ← N(e, parent, root);  H(P,s) ← (i+1, h');  U ← U ‖ (P, s, i+1, h', t)
3  return R
```

- *Compare-and-swap:* applies only if its parent is the head. Heads never repeat (Thm 3) ⇒ a signed update applies **at most once**, even if the repo returns to an old state.
- *Epoch window:* up to W epochs of inclusion lag; a late update is invalid, the publisher re-commits. Whoever orders can delay, not make a late update land.
- *No gate, no fetching:* any author commits to its own repos at its own cost; Apply computes the head from the commit's fields. Only P can make (P, s) unreadable.
- **Nothing else lives in the registry.** What an app presents, what passed a rule, what was withdrawn: curators' and validators' data (§2).
- **Cost:** one map entry + one log append; guard = one lookup, one name, one epoch.

The registry at t. R_t = Apply folded over $\mathsf{Read}_\iota(t)$. $(pos_t(\rho), H_t(\rho)) = \mathcal{H}(\rho)$. **Validity is a function of the ordering alone.** A repo's **history** = heads $h_1, h_2, \ldots$ its valid updates set.

Global state.

$$\mathcal{G}_t(\rho, k) = M_{H_t(\rho)}(k)$$

What was written, nothing else; no reading depends on an app. Final t ⇒ every honest party gets the same map. Needs no root: a checkpoint at t names it.

---

4. Directories and the update log

Def 3.2 (Directory). A state whose records are (e, c), not both $\perp$: e a repo entry (head + position), $c = \mathsf{Com}(D')$ a subdirectory. Directories nest; a path is a sequence of keys.

- **Publisher directory:** $D_{P,t}(s) = \big(pos_t(\rho), H_t(\rho)\big)$, root $\mathsf{dir}_{P,t}$.
- **No directory of every publisher** (everyone would choose its keys ⇒ shape attacks). Many publishers ⇒ via an app's citations or a lens over $\mathcal{U}$.
- Repo names may be paths $s = (s_1, \ldots, s_d)$, compared component-wise ⇒ prefixes are intervals (folders); lookup ≤ $(d+1)(h+1)$ nodes.

Lemma 3.3 (Directories only grow). $t \le t' \Rightarrow pos_t(\rho) \le pos_{t'}(\rho)$; equal position ⇒ equal entries. ⇒ merge by max.

Update log $\mathcal{U}_t$: (P, s, i, h, t_u) per valid update, root $\mathsf{log}_t = \mathsf{LCom}(\mathcal{U}_t)$. Directories are a fold of it.

Lemma 3.4 (The log locates every head). Exactly one entry per (ρ, i), $1 \le i \le pos_t(\rho)$, in increasing position; clocks non-decreasing.

Prop 3.5 (Citations are checkable). For (ρ, i, h, k): take $(j, (\rho, i, h', t_u), \pi)$ from any source, require $\mathsf{Verify}(\mathsf{log}_t, j, \cdot, \pi)$; **true** iff $h' = h$. Then $\mathsf{Get}(h)$ and open its state on $[k, k]$. True ⇒ exactly $M_h(k)$; false ⇒ ρ never had h at i. Error ≤ $\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}} + \mathbf{Adv}^{\mathsf{bind}}_{N} + \mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}$. *Idea:* position binding + L3.4 ⇒ h' is the head at i; Thm 1 + opening binding ⇒ the record. Cost: 1 log opening ($\le 2\lfloor \log_2 M \rfloor$ names), 1 commit, $O(h+1)$ nodes, 1 record. A citation is a fact about history: later updates never falsify it. Newer version? compare i with $pos_t(\rho)$. False citations are visible to every reader.

Cost per valid update: ≤ $2(h+1)$ node rewrites per directory tree on its path (L6.1); log append < 2 hashes avg, ≤ $\lfloor \log_2 M \rfloor$ (L6.3).

---

5. Reading

Def 3.6 (Selector, slice). S = finite union of boxes $[P, P] \times I_s \times I_k$ (one publisher each). $\mathcal{G}_t|_S$ = union of $\mathcal{G}_t$ restricted to each box. Examples: one record; one repo; one publisher; **an app** = its curator's repo, then each cited record, checked by Prop 3.5. (Intervals because openings are on intervals; slicing by field = slice of an index view.)

Def 3.7 (Authenticated directories & log). Light reader holds checkpoint d at final t:

1. $\mathsf{Dir}(d, B)$ for $B = [P, P] \times I_s$: $\{(P, s) \mapsto D_{P,t}(s)\}$, every repo in B with a head, nothing else;
2. $\mathsf{Log}(d) = \mathsf{st}(\mathcal{U}_t)$ ⇒ length and root; check openings, append to follow.

Wrong non-fail answer w.p. ≤ $\mathbf{Adv}^{\mathsf{dir}}$ (= 0 for a full reader).

$\mathsf{ReadSlice}(d, S)$

```
1  Z ← ∅
2  for each box [P,P]×I_s×I_k of S:
3    E ← Dir(d, [P,P]×I_s)
4    for each (P,s) ∈ dom E  (parallel):
```

```
5      h ← head of E(P,s);  (e,parent,root) ← Get(h)
6      (D',π) ← opening of root on I_k;  require Check(root,I_k,D',π)
7      for k↦n ∈ D':  Z((P,s),k) ← Get(n)
8  return Z
```

Cost (r repos, m records): r commits; $r(h+1) + m/(2^b - 1)$ nodes expected ($r(h+1) + mh$ worst); m records; **4 round trips** regardless of size.

History. A repo's *track* = its entries in $\mathcal{U}$, indices $1..n$ in order, clocks non-decreasing. One position: 1 opening. Whole history of p positions: p openings.

$\mathsf{LastAt}(\ell, n, \sigma, t')$ — last entry of track σ with clock ≤ t'

```
1  if n = 0: return ⊥
2  (j,l,π) from any source; require Verify(ℓ,j,l,π) ∧ l ∈ σ with index i, value v,
clock t_u
3  if t_u > t': require i = 1; return ⊥
4  if i < n: (j',l',π') ; require Verify(ℓ,j',l',π') ∧ l' ∈ σ, index i+1, clock > t'
5  return v
```

$\mathsf{ReadAt}(d, \rho, t')$: ℓ from $\mathsf{Log}(d)$, n from ρ's entry in $\mathsf{Dir} \Rightarrow H_{t'}(\rho)$. No old chain state needed.

Prop 3.8 (History reads sound & complete). LastAt errs w.p. ≤ $2\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$; ReadAt ≤ $2\mathbf{Adv}^{\mathsf{dir}} + 2\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$; an honest source never fails. *Idea:* position binding + L3.4 ⇒ entry i at/before t' and entry $i+1$ after ⇒ i is last.

Follow (keeping up with changes). Cursor (B, I_k, st, E).

1. Fetch log entries $M+1..M'$, append to st, require $\mathsf{Root}(st) = \ell'$ ⇒ every new entry, in order.
2. Replay the entries for repos in B: new heads.
3. For each changed repo, Diff old and new state trees: skip equal-named subtrees and subtrees outside I_k.

Cost: $\Delta_{\mathcal{U}}$ entries, ≤ $2\Delta_{\mathcal{U}} + \lfloor \log_2 M \rfloor + 1$ hashes; $2c'$ commits (c' changed repos); $O((\Delta + c')(h+1))$ nodes; Δ records. **Independent of slice size.**

- **Following an app** = following its curator's repo; a changed citation costs one Prop 3.5 check, an unchanged one nothing.
- **Ceiling:** following one publisher alone reads all of $\mathcal{U}$ (~80 B/entry) — or an indexer's sub-log (§8).

---

6. Security of data and history

Thm 1 (Binding). Reader accepts only M_h for head h, except $\mathbf{Adv}^{\mathsf{bind}}_N + \mathbf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}$.

Thm 2 (Authenticity). Every head of (P, s) was set by an entry authored by P. *(guard + Authorship.)*

Thm 3 (Linear history). Valid updates of a repo form one sequence; the i-th has parent h_{i-1}; no head repeats except $\mathbf{Adv}^{\mathsf{bind}}_N$. *Idea:* least j with $h_i = h_j$: equal commits ⇒ $h_{i-1} = h_{j-1}$, contradiction; unequal ⇒ collision. ⇒ a signed update applies at most once. The registry computes heads ⇒ the order and the parent chain are **one history**.

Thm 4 (Slice soundness). t final, q boxes:

$$\Pr[\mathsf{ReadSlice}(d, S) \notin \{\mathcal{G}_t|_S, \mathsf{fail}\}] \le q\,\mathbf{Adv}^{\mathsf{dir}} + \mathbf{Adv}^{\mathsf{bind}}_{N} + \mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}.$$

Idea: line 3 true by Dir; line 5 by name; line 6 by opening binding; line 7 by name. **Completeness, not just inclusion:** hosts can withhold ($\Rightarrow \mathsf{fail}$), not truncate.

Cor 4.1 (Historic slices). Line 3 at t + Log + LastAt per repo + lines 5–7 $\Rightarrow \mathcal{G}_{t'}|_S$, except $2q\,\mathbf{Adv}^{\mathsf{dir}} + 2r\,\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}} + \mathbf{Adv}^{\mathsf{bind}}_{N} + \mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}$ (r repos).

Cor 4.2 (Reading an app). Thm 4 on the curator's repo + Prop 3.5 per citation ⇒ exactly the citations committed at t, and for each true one, exactly the record its publisher committed. **An app chooses what to present, never alters it.**

---

7. Lenses (views)

Def 3.9 (Lens). $L = (S, f)$, f deterministic slices → states. View $V_L(t) = f(\mathcal{G}_t|_S)$, root $\mathsf{Com}(V_L(t))$; lens named $N(\texttt{lens}, L)$. A view is a state ⇒ sliceable, openable, composable, committable as a repo. A lens may take an app's citations as input (each checked by Prop 3.5) ⇒ as trustworthy as the curator's choices.

Prop 3.10 (Views are determined). All honest parties get the same root. *(Thm 4 + determinism + unique representation.)* No producer, nothing to vote on.

Def 3.11 (Pointwise). $\exists\,\tau, g$: $\tau(x, r)$ injective in x, $g(r) \in Rec \cup \{\bot\}$,

$$f(Z) = \{\tau(x, r) \mapsto g(r) : (x \mapsto r) \in Z,\ g(r) \neq \bot\}$$

with $x = (P, s, k)$.

Prop 3.12 (Pointwise update). If Z, Z' differ on keys X: delete $\tau(x, Z(x))$, then insert $\tau(x, Z'(x)) \mapsto g(Z'(x))$. Cost $2|X|$ ops × $2(h+1)$ nodes ⇒ follows its slice at $O((\Delta + c')(h+1))$.

lens	$\tau(x, r)$	$g(r)$	notes
filter	x	r if type θ else $\bot$	pointwise
validation	x	r if $\varphi(r)$ else $\bot$	read-time check, anyone; single-record rules only
curation	x	r unless x in a repo of $\mathcal{X}$	$\mathcal{X}$ may be read from a curator's repo
index on a	$(r.a, x)$	r	value v = one opening, $O(h + j)$; top-j = last j keys
embedding	x	$E(r) \in \mathbb{R}^d$	E must be bit-deterministic (fixed-point, quantized)
sub-log (over $\mathcal{U}$)	$(\ell.P, j)$	ℓ	P's updates form an interval
aggregate	—	—	not pointwise, incremental $O(\lvert X\rvert)$
join / general	—	—	recompute; no succinct fraud proof

Trusting a view: recompute (trust only Thm 4) · accept from a host, check later (App. C makes it accountable) · check a proof of the view (open).

8. Indexers and search

Indexer = follows the global state, maintains views, serves them. Like an AppView/labeller but **no authority** (input public, function named) and **completeness checkable** (the on-chain update log).

Def 3.13 (Claim). Signed $c = (\iota, N(\texttt{lens}, L), t, R)$ asserting $R = \mathsf{Com}(V_L(t))$ (log lenses: $t \to M$).

Discovery. $\mathcal{U}$ names every repo with a head ⇒ "active publishers", "repos of P", "apps" (repos holding a manifest) are lenses over the log; "apps citing ρ", "repos a validator passed" are index lenses over citations.

Following one publisher via a sub-log claim: 1 slot proof (log length M), 1 signature, $O(h + \Delta_P)$ nodes. Forged entries caught by $O(\log M)$ checks; omissions by cross-checking or fraud proofs (App. C).

Trust ladder: trust → hold to account (App. C, one honest watcher) → cross-check k indexers → verify proof of view → recompute. Lag is visible (M' vs M); refusal ⇒ switch or recompute.

Score $\langle q, v\rangle$, $\|q\| \le 1$. Three ways to serve, in order of preference:

Bundle (default for small views). The embedding view sharded by key interval; each shard an opening of the claimed root. Reader fetches all, ranks locally ⇒ **exact and complete** (no forged *or omitted* entry); hosts learn which views, never the query. ~26 MB for 10^5 × 256-d int8; then follow by change.

Bounded tree (large views). Def 3.14. Leaves $(x, v) \in V$; internal nodes $(\mu, \varepsilon, \text{children})$. **Sound** iff every leaf v below has $\|v - \mu\| \le \varepsilon$. Bound: $\beta(n) = \langle q, \mu\rangle + \varepsilon$ (internal), $\langle q, v\rangle$ (leaf).

$\mathsf{Search}(\alpha, q, j)$ — best-first: pop max-β node; stop if $|T| = j$ and $\beta(n) < \min_T \langle q, v\rangle$; leaves → keep top j; internal → push children.

Lemma 3.15 (Exact pruning).

$$\langle q, v\rangle = \langle q, \mu\rangle + \langle q, v - \mu\rangle \le \langle q, \mu\rangle + \|q\|\,\|v - \mu\| \le \langle q, \mu\rangle + \varepsilon$$

⇒ on a sound tree, Search returns the exact top-j. Quantization absorbed into ε.

Not a view (indexer picks the shape); checked by soundness: false bound / forged leaf have short proofs (App. C), **omission: open**. Insert $O(depth \cdot d)$; delete keeps soundness. Host sees which nodes are expanded (rough query location).

Answering queries — a downgrade: the indexer sees every query.

9. Instantiation

Ordering = blockchain. Blocks = sequence; timestamps = clock; consensus = Verify. Registry = a contract running Apply: stores $\mathcal{H}$, and for $\mathcal{U}$ only MMR peaks + length, plus each publisher's repo list. ι = chain id + address (EIP-712 domain). The contract computes $h' = N(e, parent, root)$ itself (SHA-256 precompile, a few hundred gas) and rejects late updates with its own error.

- **Curators** are authors: a key, or a contract (moderators' threshold, a validator's verdict). An optional **naming program** maps short app names → curator repo + payout address; outside the registry, decides

nothing about data.

- Dir by **storage proofs** (publisher → repo list, head + position slots; no prover). List is in creation order ⇒ a narrow I_s pays for all of P's repos.
- **Gas** (estimates): update ~20k (4 existing slots), first write ~100k. Log: 2 writes per entry (one peak + M); no root stored (writes paid every update, reads only by readers).
- Finality δ: ~1h full node on rollup; ~7 days for trustless light reader on optimistic rollup (needs archive state).

Com = **Merkle Search Tree.** $\mathrm{level}(k) = \lfloor \mathsf{lz}(N(k))/b \rfloor$, fanout 2^b. Level-ℓ node = keys of level exactly ℓ in a *gap* between consecutive keys of level $> \ell$ ⇒ tree is a function of the map (unique representation). Height h = max level; $h \approx \log_{2^b} n$; $h \le 2\lambda/b$. **Well-formed:** sorted, inside gap, level exactly ℓ, ≤ $B_{\max}$ keys.

Lemma 6.0 (Honest nodes small; ROM). $\Pr[\text{node} \ge m \text{ entries}] \le (1-2^{-b})^m$; $b = 4$, $B_{\max} = 1024$: $\approx 2^{-95}$. *(geometric run length)*

Shape attacks: height ℓ costs ~$2^{b\ell}$ tries; filling a node is free but $B_{\max}$ makes it *malformed* ⇒ hurts only the key-chooser's readers ⇒ **one key-chooser per tree** (publisher: record keys, repo names; curator: its citations' keys).

Lemma 6.1 (Locality). Maps differing on Δ keys share all but ≤ $2\Delta(h+1)$ nodes per tree. *(a key lies in the closed gap of ≤ 2 nodes per level)* ⇒ update rewrites ≤ $2(h+1)$; diff-walk $O(\Delta(h+1))$.

Lemma 6.2 (Openings). Opening on I (holding m keys) = nodes whose gaps meet I; count

$$\le (h+1) + \sum_{k \in D|_I} \mathrm{level}(k) \;\le\; (h+1)(m+1), \qquad \mathbb{E} = (h+1) + \tfrac{m}{2^b-1}.$$

Opening binding by induction over reached nodes (well-formedness confines keys to gaps).

LCom = **Merkle Mountain Range.** Leaf $N(\texttt{leaf}, l_j)$; node $N(\texttt{node}, L, R)$; $M = \sum 2^{e_i}$ ⇒ perfect trees, roots = peaks; root = $N(\texttt{mmr}, M, peaks)$; $\texttt{st} = (M, peaks)$. Append = add 1-leaf tree, merge equal-size rightmost. Shape fixed by M ⇒ ungrindable. (On-chain: keccak over `abi.encode`, never `encodePacked`.)

Lemma 6.3 (MMR). (1) binding & position binding ⇐ N binding (tags separate leaf/node); (2) ≤ $\lfloor \log_2 M \rfloor + 1$ peaks (set bits of M); (3) opening ≤ $2\lfloor \log_2 M \rfloor$ names; (4) Δ appends ≤ $\Delta + \lfloor \log_2 M \rfloor$ merges ⇒ < 1 merge avg, ≤ $\lfloor \log_2 M \rfloor$ each.

Operators (§7). The protocol decides what is committed, nothing about content. Hosts, indexers, curators and key releasers [WP2 §4.3] each apply their own policy, and each refusal is visible or detectable.

---

10. Appendix A — agents and the semantic CDN (design)

- **One server for every app:** list apps (lens over $\mathcal{U}$ for repos with a manifest) → mount views (checked against a claim's root) → same tools on every view (describe, search, get, similar, count, browse). Works because every view is the same kind of state.
- **Semantic CDN** = route by meaning, deliver by name; routing is checkable, not trusted.

Def A.1 (Clustered view). Model E, fixed centres $C = (\mu_1, \ldots, \mu_m)$ named in the lens:

$$\tau(x, r) = (\kappa(r), x), \quad g(r) = E(r), \quad \kappa(r) = \text{nearest centre (lowest index on ties)}$$

Pointwise + key-revealing ⇒ **a cluster = one interval = one opening, complete** (fixes the bounded tree's omission gap); misfiled / wrong / omitted entry ⇒ fraud proof (Prop C.2); follows by change. New centres ⇒ new lens.

Def A.2 (Coverage). Claim + radii: $\mathsf{cov} = (\iota, N(\texttt{lens}, L), t, R, (\varepsilon_i))$. Sound iff every v in cluster i has $\|v - \mu_i\| \le \varepsilon_i$. Bound $\beta_i(q) = \langle q, \mu_i \rangle + \varepsilon_i$. A few KB per view.

Prop A.3 (Checkable). Unsound ⇒ one opening at one entry proves it; sound ⇒ $\langle q, v \rangle \le \beta_i(q)$ (L3.15). Loose-but-honest radius = cost, not fraud.

$\mathsf{Route}(q, j)$ — best-first over every cluster of every view by $\beta_i(q)$; stop when $|T| = j$ and next bound < j-th best; else open cluster (one checked opening from any host), keep top j.

Prop A.4 (Exact & complete). On sound coverages: exact top-j across all views; opens only clusters with bound ≥ the final j-th score. *(opening binding + A.3)*

Recursion. Directory = a view of clusters keyed $(N(\texttt{lens}, L), i)$ with value (μ_i, ε_i); cluster it again with upper radius

$$\delta = \max_i \left(\|\mu_i - \nu\| + \varepsilon_i \right)$$

(triangle inequality ⇒ sound if lower levels are) ⇒ one tree of clustered views: upper levels = views, lower = records. Many directories, each a curation (e.g. every view some app cites, every repo a validator passed).

Guarantees: no authority · exactness · privacy (query local; hosts see clusters opened) · delivery by name from any host. **Cost:** nm inner products to route ($10^4 \times 8 \approx$ ms), then one opening per cluster opened.

- **Cartridge** = (lens name, t, root, fetched clusters/shards): hot-swappable, composable (same E ⇒ comparable scores), pinned (answers cite root + t), checked, followed by change. Agents publish their own memory as repos ⇒ cartridges owned by their key.
- **Open:** one bit-exact E per CDN (cross-model mapping not given); choosing centres / reclustering / radius tightness (cost only); cluster counts (aggregate, no succinct fraud proof); leakage from clusters opened; sealed views cover public fields only [WP2 §2.3].

---

11. Appendix B — micro-commits between anchors (design)

Def B.1 (Micro-commit). On base i of ρ: P-signed $\mu = (\iota, \rho, i, j, a, root, \tau)$, $j \ge 1$ its index, $a = \mathsf{LCom}(\mu_1..\mu_{j-1})$, τ informational. Sub-log L_i; **soft head** $(i, j, N(\mu_j))$. A commit that is never submitted; linked by a log commitment ⇒ membership is one opening.

Def B.2 (Anchor). Update with commit $\gamma = (e, h_i, root, a)$, $a = \mathsf{LCom}(\mu_1..\mu_k)$, $root$ of μ_k; head $N(e, h_i, root, a)$ ($a = \perp$ ⇒ ordinary commit). Registry change: hash a in when present; guard unchanged. Thm 1 and Thm 3 hold unchanged.

Def B.3 (Faults). F1 double signing (same j, two values) · F2 omission/substitution (anchor's sub-log shorter than j, or opens to another value at j) · F3 wrong root.

Prop B.4 (Pre-confirmations bind). A signed μ_j is at index j of the final anchor at $i{+}1$, or an F2 is checkable with one opening; two holders of index j agree or hold an F1. ⇒ unanchored data is **sellable**. Not given: currency, liveness, availability.

- **Accountability:** faults convince anyone; optional bond pays the first fault. **Forced inclusion** (posted μ, bond pays if no anchor by $t_\mu + \Theta$, $\Theta \geq W$) [no proof].
- **Cost:** micro-commit = 1 signature + < 2 hashes + $O(\Delta(h+1))$ nodes, no entry; anchoring every Θ s costs $\propto 1/\Theta$ **whatever the rate of change**; soft head final within $\Theta + \delta$. Relayed anchors: EIP-712-signed updates, anyone submits.
- **Streams** sealed per epoch are sold by the epoch [WP2 §2.2]. **Fleets:** one repo per device, $s = (\mathit{fleet}, \mathit{device})$; delegated device keys (unspecified).

---

12. Appendix C — accountable indexers

Point read at $x = (P, s, k)$ *as of* t: ReadAt + lines 6–7 of ReadSlice on $[k,k] \Rightarrow \mathcal{G}_t(P,s,k)$. (As of t, so later updates can't make honest claims "wrong".) Over a log: positional opening, or $\perp$ if $j > M$.

Def C.1 (Key-revealing). Pointwise and x computable from $\tau(x, r)$. (Filter, validation, curation, index, sub-log, embedding, clustered.)

Fraud proof $\mathit{fp} = (y, V', \pi, x, p)$. CheckFraud$(c, \mathit{fp})$

```
require Check(R,[y,y],V',π)                      ▷ V' = ∅ or {y↦v}
if V' ≠ ∅: x ← key revealed by y
z ← value at x from point read p; require p checks as of t
U ← {y↦g(z)} if z≠⊥ ∧ g(z)≠⊥ ∧ τ(x,z)=y, else ∅
return U ≠ V'                                     ▷ true ⇒ claim wrong at y
```

Prop C.2 (Succinct fraud proofs). L pointwise & key-revealing.

1. **Completeness:** $R \neq \mathsf{Com}(V_L(t))$ (well-formed inputs) ⇒ $\exists\, \mathit{fp}$ accepted; 1 opening + 1 point read + one eval of τ, g.
2. **Soundness:** $R = \mathsf{Com}(V_L(t))$ ⇒ no accepted fp, except opening binding / N binding / Prop 3.8. Holds for claims of **any age**.

Idea: $U = V|_{[y,y]}$ whenever z is true and ($y = \tau(x,z)$ or $y \notin V$). Completeness: pick y in the symmetric difference. Soundness: opening binding forces $V' = V|_{[y,y]} = U$.

Cost: log lens $O(h+1) + 2\lfloor \log_2 M \rfloor$; slice lens: map opening + 2 log openings (head) + 1 state opening. **Equivocation:** two roots, same (L,t) ⇒ ≥1 wrong ⇒ fraud proof. Unlike [TG]'s proof of indexing: the claim commits to the view, so a mismatch walks down to one key, **no arbiter**.

Watchers [no proof of sufficiency]: someone recomputes (other indexers; a publisher for its own sub-log; anyone paid by a bond) · a watcher must see the root (cross-check, or published claims) · the reader can complete the proof from nodes it already read · **no challenge window**.

Bounded trees: false bound = path to leaf with $\|v - \mu\| > \varepsilon$; forged leaf = path + point read of the embedding view; omission: open.

Bonds: pay the first valid fraud proof; log lenses check against current peaks (proofs stay valid forever); slice-lens on-chain cost not given. **Other lenses** (aggregates, joins, general): need the input they read; open.

---

13. Open / unproven

Bit-deterministic embeddings (assumed) · epoch window delay · bounded-tree omissions, search cost, query leakage · nested-directory interval bound · whole-history openings sharing a tree · joins · expected costs assume random names · gas/proof sizes (estimates) · light-client protocol (assumed) · non-pointwise fraud proofs · watcher sufficiency & collusion · sub-log address/index grinding · proofs of views · validators' evidence (proofs, attestations) · curators as programs · micro-commits: forced inclusion, device keys, sub-epoch sealing, F2/F3 gas · sealed data, read conditions, referral payment (WP2).